

Sonstige Themen

Willkommen!

Sämtliche Notizen sind nur zur Unterstützung für den Trainer gedacht.



Operatoren überladen

- Unäre Operatoren (+, -, !, ~, ++, --, true, false)

```
public static Zug operator ++(Zug wagon)
```

- Binäre Operatoren (+, -, *, /, %, &, |, ^, <<, >>)

```
public static Zug operator +(Zug wagon, Wagon wagon)
```

```
public static Zug operator +(Wagon wagon, Zug zug)
```

```
public static Zug operator +(Wagon wagon1, Wagon wagon2)
```

- Vergleichsoperatoren (==, !=, <, >, <=, >=)

```
public static bool operator ==(Wagon wagon1, Wagon wagon2)
```

2

- Die Operatoren müssen immer als statische Methode einer Klasse deklariert werden.
- Mindestens einer der Parameter muss vom Typ der Klasse sein, in der die Methode deklariert ist
- Die Reihenfolge der Parameter bei binären Operatoren ist wichtig
- Bei Vergleichsoperatoren muss der Gegenoperator auch mit definiert werden (== und !=, < und >...)



Typecasting überladen

```
public class CastList<T> : List<T>
{
    public static implicit operator T[](CastList<T> list)
    {
        return list.ToArray();
    }

    public static explicit operator T[](CastList<T> list)
    {
        return list.ToArray();
    }
}

CastList<int> cl = new CastList<int>();
int[] arrayImplizit = cl; //Implizite Konvertierung möglich
int[] arrayExplizit = (int[]) cl; //Explizite Konvertierung möglich
```

3

- Die Operatoren müssen immer als statische Methode einer Klasse deklariert werden.
- Mindestens einer der Parameter muss vom Typ der Klasse sein, in der die Methode deklariert ist
- Die Reihenfolge der Parameter bei binären Operatoren ist wichtig
- Bei Vergleichsoperatoren muss der Gegenoperator auch mit definiert werden (== und !=, < und >...)



Enumerator für eine Klasse definieren

```
public class Zug : IEnumerable           //Interface implementieren
{
    public List<Wagon> Wagons { get; set; }

    IEnumerator IEnumerable.GetEnumerator()
    {
        foreach(var item in Wagons)
        {
            yield return item;
        }
    }
}

Zug ICE = new Zug("ICE");
foreach(Wagon item in ICE)
```

4

- Foreach ruft automatisch die Methode GetEnumerator einer Klasse auf, wenn diese das Interface IEnumerable implementiert
- Über das Schlüsselwort yield wird ein Wert für einen Schleifendurchlauf zurückgegeben
- Beim nächsten Schleifendurchlauf wird die Methode an der Stelle der zuletzt ausgeführten yield-return-Anweisung fortgesetzt
- „yield break“ beendet den Iteratoren-Prozess



Indexerproperties

```
public class Zug
public List<Wagon> Wagons { get; set; }

public Wagon this[int index] //Indexer-Property
{
    get { return this.Wagons[index]; }
    set { this.Wagons[index] = value; }
}

Zug ICE = new Zug("ICE");
Wagon SpeiBewagen = ICE[3]; //Zugriff erfolgt wie bei Array
```



Anonyme Objekte

- Schnell und einfach Variablen zwischenspeichern
- Typen werden automatisch erkannt
- muss auf eine Variable vom Typ `var` zugewiesen werden
- besonders nützlich mit Linq

```
var x = new { ID = 1, Name = "Max" };  
Console.WriteLine($"{x.ID}, {x.Name}");
```



StringBuilder

- Strings zusammensetzen ohne +
- Wesentlich performanter als +
- fertigen String erhalten mit ToString()

```
StringBuilder sb = new StringBuilder();  
sb.Append("Test1");  
sb.AppendLine("Test2");  
Console.WriteLine(sb.ToString());
```